Assignment 2 : Simple Search Engine using Hadoop MapReduce

Ekaterina Akimenko
e.akimenko@innopolis.university

1 Introduction

The goal of this project is to build a simple search engine using distributed technologies. The main idea was to take a large text dataset and process it step by step using different tools, each one responsible for its own part.

The system builds an inverted index using Hadoop MapReduce, then computes additional statistics using Spark, stores everything in Cassandra. It allows the user to run search queries using BM25 ranking.

2 Methodology

2.1 Dataset Preparation

The dataset consists of text documents, where each file represents one document. The naming format is important, because it helps us later extract both the document id and the title. Each file follows this structure:

```
<doc_id>_<doc_title>.txt
```

At the beginning, all documents were merged into a single file and uploaded to HDFS. The final location was:

```
/input/data
```

This step is needed so Hadoop can process the data properly in a distributed way. Instead of working with many small files, it works with one combined dataset.

2.2 Index Construction using Hadoop MapReduce

The main indexing step is done using Hadoop Streaming. I used a Python mapper and reducer, which makes it easier to control the logic.

The mapper reads each document, tokenizes the text, and converts everything to lowercase. The tokenization is done using a simple regex, so it captures words and numbers. It is not perfect, but it is simple and fast, which was the goal.

For each document, the mapper counts how many times each term appears. This gives us the term frequency (TF). Then it emits intermediate results that are grouped by term.

The reducer receives all values for each term and combines them. It builds the postings list, which basically tells us in which documents the term appears and how many times. At the same time, it also stores document length and title information.

After this stage, we already have a raw inverted index stored in HDFS.

2.3 Post-processing with Spark

After MapReduce, Spark is used to organize and improve the index.

First, the output is split into separate parts, so that different types of data are stored independently. We separate documents, postings, vocabulary, and statistics. This makes the system cleaner and easier to work with.

Then, Spark is used to compute document frequency (DF) for each term. This is important for ranking later.

Finally, global statistics are calculated. These include:

- total number of documents (N)
- average document length (AVGDL)

All results are stored in HDFS in structured folders:

```
/indexer/docs
/indexer/postings
/indexer/vocab
/indexer/stats
```

2.4 Storage in Cassandra

After everything is computed, the index is loaded into Cassandra. The idea here is to make querying fast and simple.

The schema is split into several tables. Documents store metadata like title and length. Vocabulary stores terms and their document frequency. Postings connect terms to documents with term frequency values. There is also a table for corpus statistics.

This structure is not too complicated, but it works well for this task. It allows quick lookups and avoids unnecessary joins.

2.5 Query Processing and Ranking

The search part of the system is based on BM25 ranking. When a query is entered, it is first tokenized in the same way as documents.

Then for each term, the system retrieves:

- postings (which documents contain the term)
- document lengths
- document frequency

After that, BM25 scores are calculated. These scores take into account how often a term appears and how long the document is. This helps balance relevance.

All scores are summed across query terms, and finally the top 10 documents are returned.

3 Demonstration

3.1 Running the System

The whole pipeline can be run using:

`/app/index.sh`

This script runs all stages one by one. It builds the index using Hadoop, processes it with Spark, and stores it in Cassandra.

After that, queries can be executed using:

`/app/search.sh <query>`

3.2 Index Creation

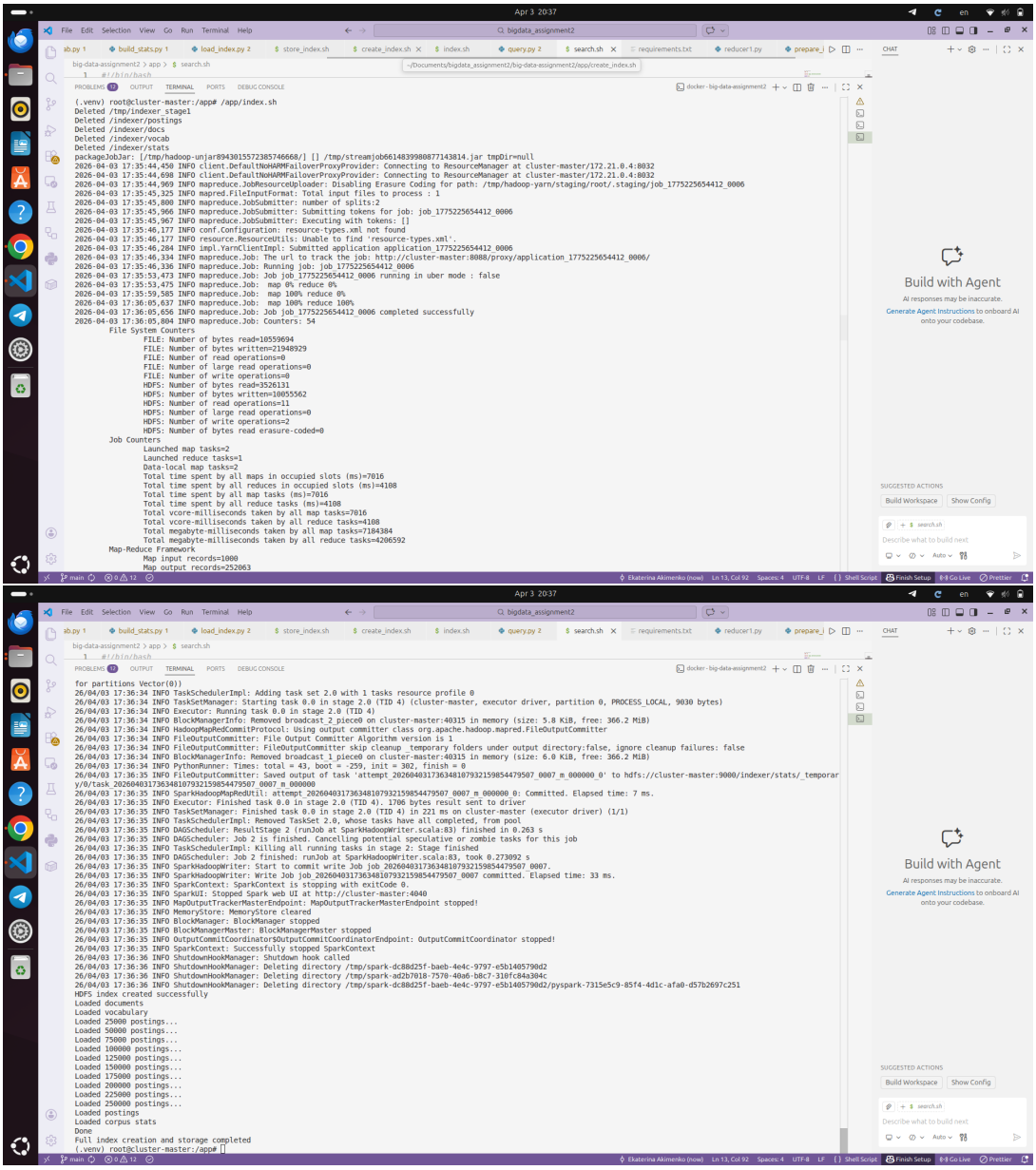

Figure 1: Successful execution of indexing pipeline

From the screenshots we can see that the indexing pipeline runs without errors. Hadoop processes the data first, and then Spark completes the remaining steps. The process is quite long, but it finishes successfully.

3.3 Index Storage in HDFS

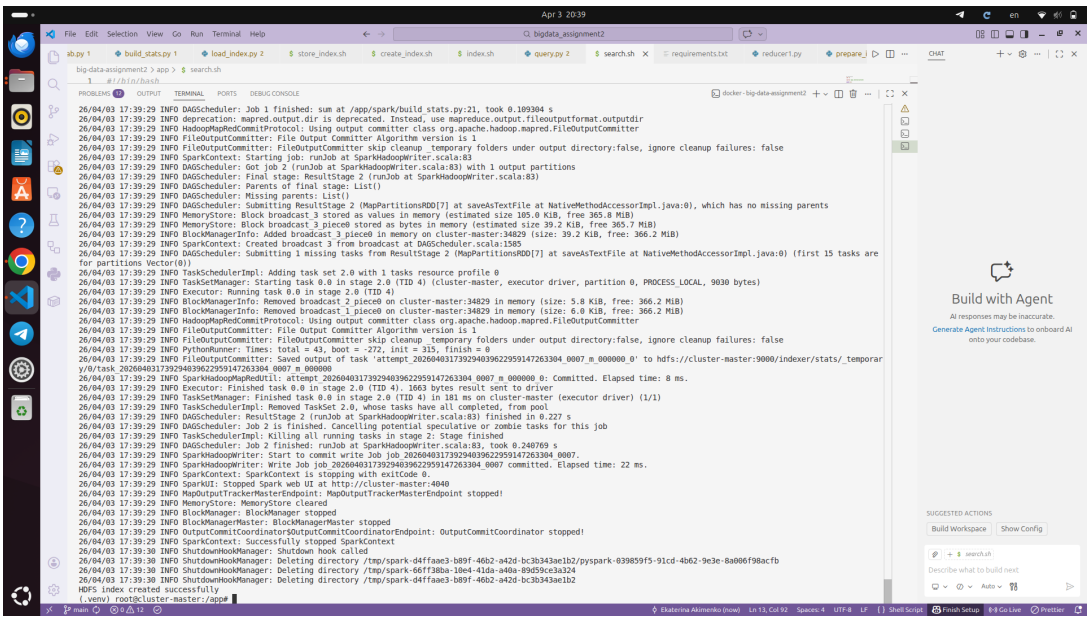

Figure 2: Index structure stored in HDFS

The index is stored in multiple folders. This separation is useful because each part has a different purpose. For example, postings are used for searching, while statistics are used for scoring.

3.4 Corpus Statistics

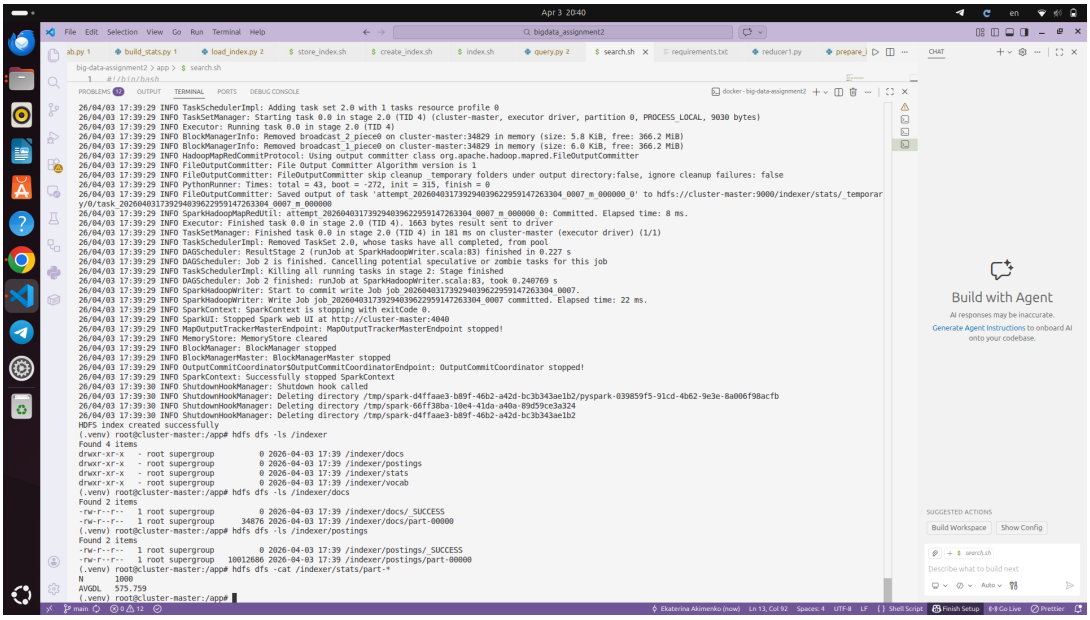

Figure 3: Corpus statistics (N and AVGDL)

Here we can see the total number of documents and the average document length. These values are later used in BM25 calculations.

3.5 Search Results

3.5.1 Query: history

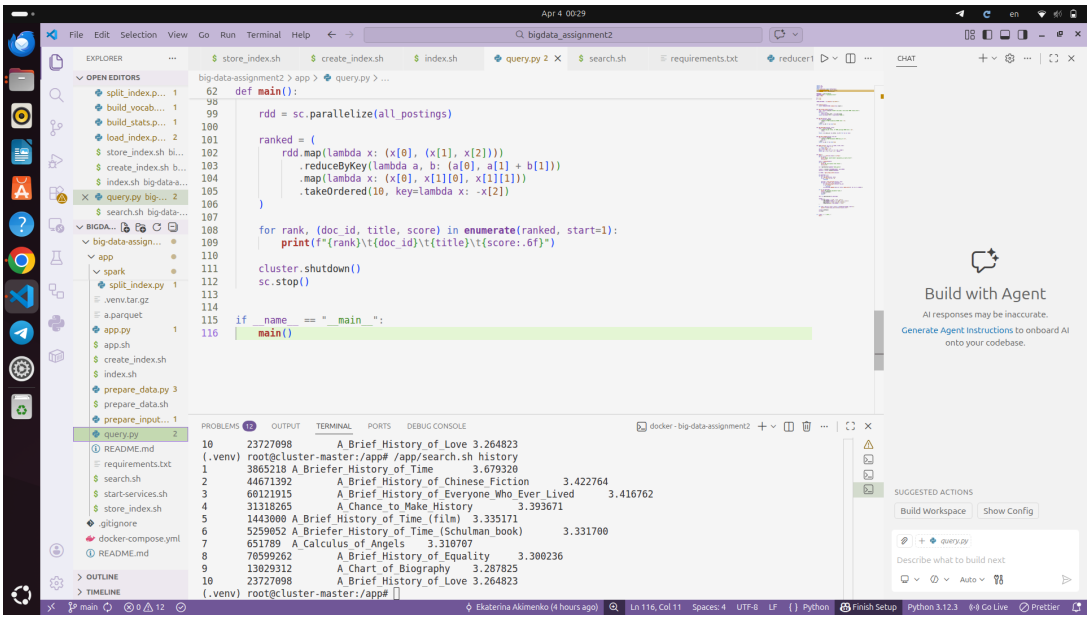

Figure 4: Search results for query: history

For this query, most of the results contain the word “history” in the title. This is expected, because BM25 gives higher scores to documents with matching terms.

3.5.2 Query: history of time

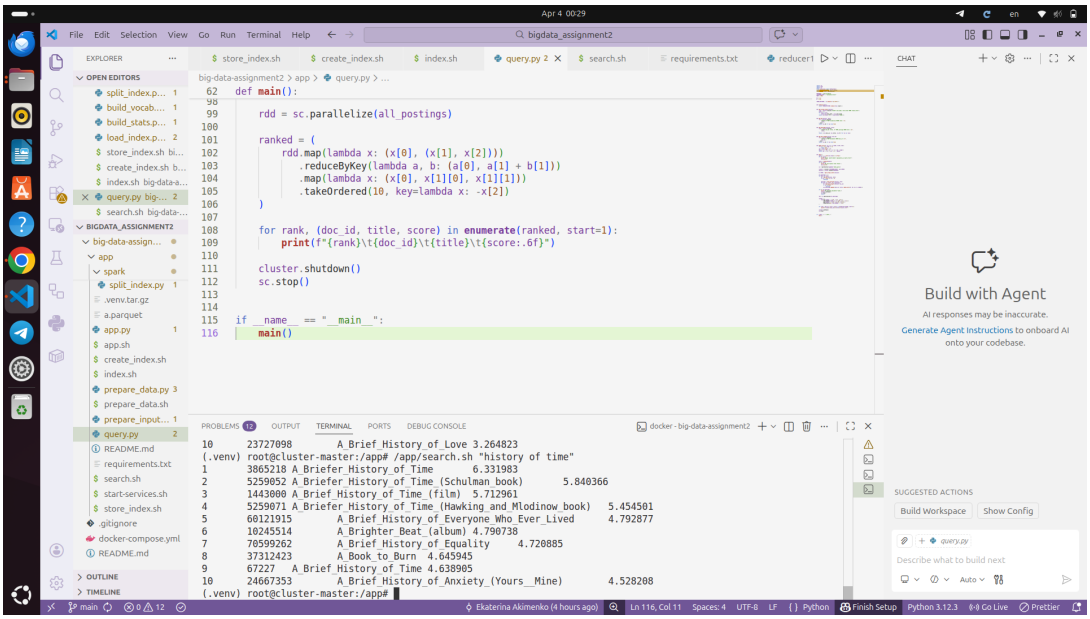

Figure 5: Search results for query: history of time

This query is more specific, and the results are also more precise. Several versions of “A Brief History of Time” appear at the top. This shows that the system correctly handles multiple terms.

3.5.3 Query: christmas

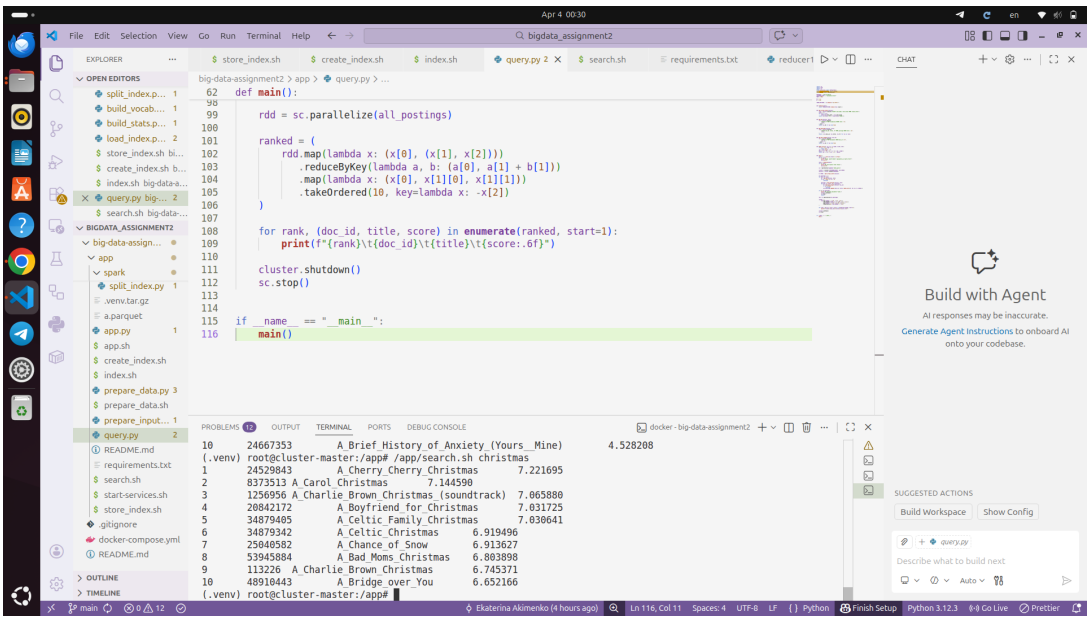

Figure 6: Search results for query: christmas

The results clearly relate to Christmas. Titles match the query well, so the ranking looks correct.

3.6 Discussion

Overall, the results look reasonable. The system returns documents that are relevant to the query. More specific queries produce better results, which is expected.

BM25 seems to work well in this setup, even though the preprocessing is quite simple.

4 Results and Discussion

The system is able to retrieve meaningful results for different queries. It performs especially well when the query matches document titles.

One interesting observation is that exact matches often appear at the top. Also, combining multiple words improves ranking quality.

The tokenizer is very simple and keeps numbers as well. This is not ideal, but it keeps the system easy to implement and still works fine.

5 Conclusion

In this project, a simple search engine was implemented using Hadoop, Spark, and Cassandra.

The system builds an inverted index, computes useful statistics, stores data efficiently, and allows ranked search queries.

Even though the implementation is not very complex, it still produces god and interpretable results. This shows that even a basic approach can work quite well when combined with distributed tools.